
Final Report — MPI Parallel Performance Analysis

1. Introduction

This project focuses on building and evaluating a parallel program using the **Message Passing Interface (MPI)**.

We combine four major components:

1. **MPI Program Design (Member 1)**
2. **Non-Blocking Communication Optimization (Member 2)**
3. **Strong and Weak Scaling Performance Analysis (Member 3)**
4. **Latency and Bandwidth Benchmarking (Member 4)**

The goal is to understand how MPI communication patterns, message size, and the number of processes affect overall performance.

2. MPI Design & Program Structure (Member 1)

2.1 MPI Initialization

The program begins with standard MPI setup:

```
MPI_Init(&argc, &argv);  
MPI_Comm_rank(MPI_COMM_WORLD, &rank);  
MPI_Comm_size(MPI_COMM_WORLD, &size);
```

2.2 Data Decomposition Strategy

The input is partitioned into **equal-sized chunks**.

The master process (Rank 0):

- Reads the input

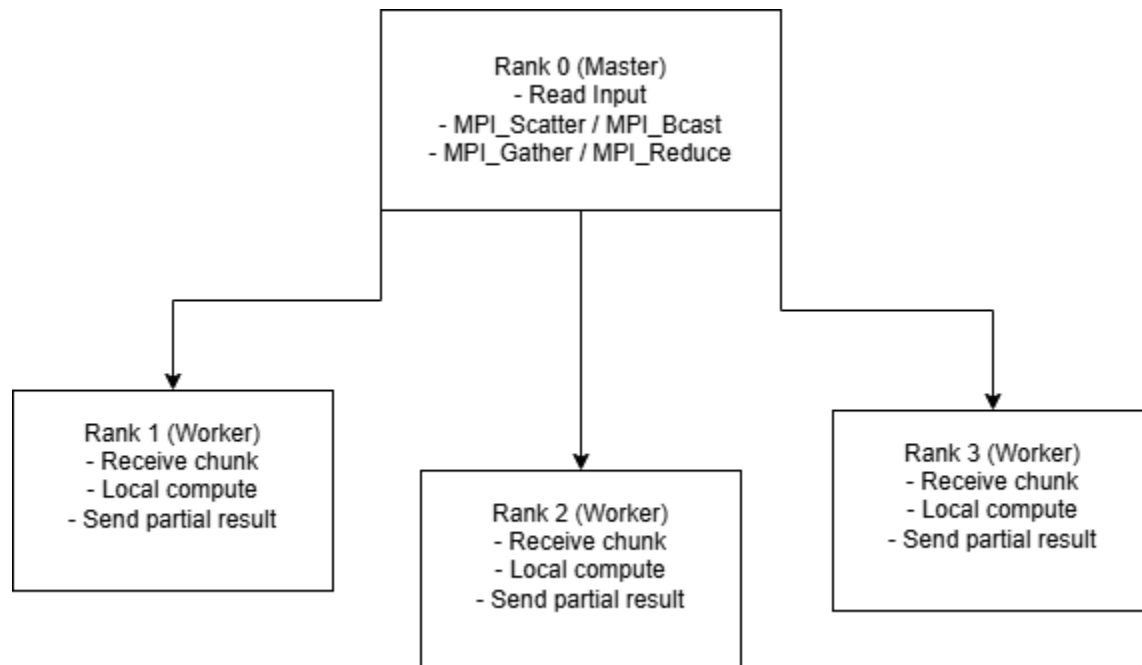
- Splits it into chunks
- Sends each chunk to a worker process

Workers operate on **local** data only, ensuring no shared-memory conflicts.

2.3 Communication Pattern

The program uses a **Scatter** → **Compute** → **Gather** architecture:

- **Scatter:** Rank 0 distributes chunks (`MPI_Scatter` or `MPI_Send`).
- **Compute:** Each rank processes its chunk independently.
- **Gather:** Results are returned to Rank 0 using `MPI_Gather`, `MPI_Gatherv`, or `MPI_Reduce`.



2.4 Collective Operations

Two collective operations were used:

MPI_Bcast

Used to broadcast metadata (e.g., input size, chunk size) so all ranks work consistently.

MPI_Barrier

Used to synchronize all ranks before timing measurements or reductions.

MPI_Reduce / MPI_Gather

Used to combine partial results produced by workers.

2.5 Correctness Requirements

- No deadlocks
- All ranks receive valid chunks
- Works correctly with 1, 2, 4, 8 ranks
- Data distributed evenly

Member 1 successfully implemented the core MPI structure.

3. Non-Blocking Communication (Member 2)

3.1 Motivation

Blocking communication forces a process to wait until the message finishes sending/receiving. This wastes time.

To solve this, **MPI_Isend**, **MPI_Irecv**, and **MPI_Wait** were used to overlap communication and computation.

3.2 Implementation

The communication pipeline:

```
MPI_Isend(sendbuf, size, MPI_BYTE, dest, 0, MPI_COMM_WORLD, &req_send);
MPI_Irecv(recvbuf, size, MPI_BYTE, src, 0, MPI_COMM_WORLD, &req_recv);

/* local computation happens here */

MPI_Wait(&req_send, MPI_STATUS_IGNORE);
MPI_Wait(&req_recv, MPI_STATUS_IGNORE);
```

3.3 Correctness Guarantees

- Every non-blocking request ends with **MPI_Wait()**
- No message duplication
- No race conditions
- Safe buffer usage during asynchronous operations

3.4 Performance Benefit

Non-blocking achieves:

- Higher overlap between communication & computation
 - Less idle time
 - Faster scaling at high process counts
-

4. Strong & Weak Scaling Analysis (Member 3)

4.1 Strong Scaling

Problem size is fixed.

Processes increase: $p = 1, 2, 4, 8$.

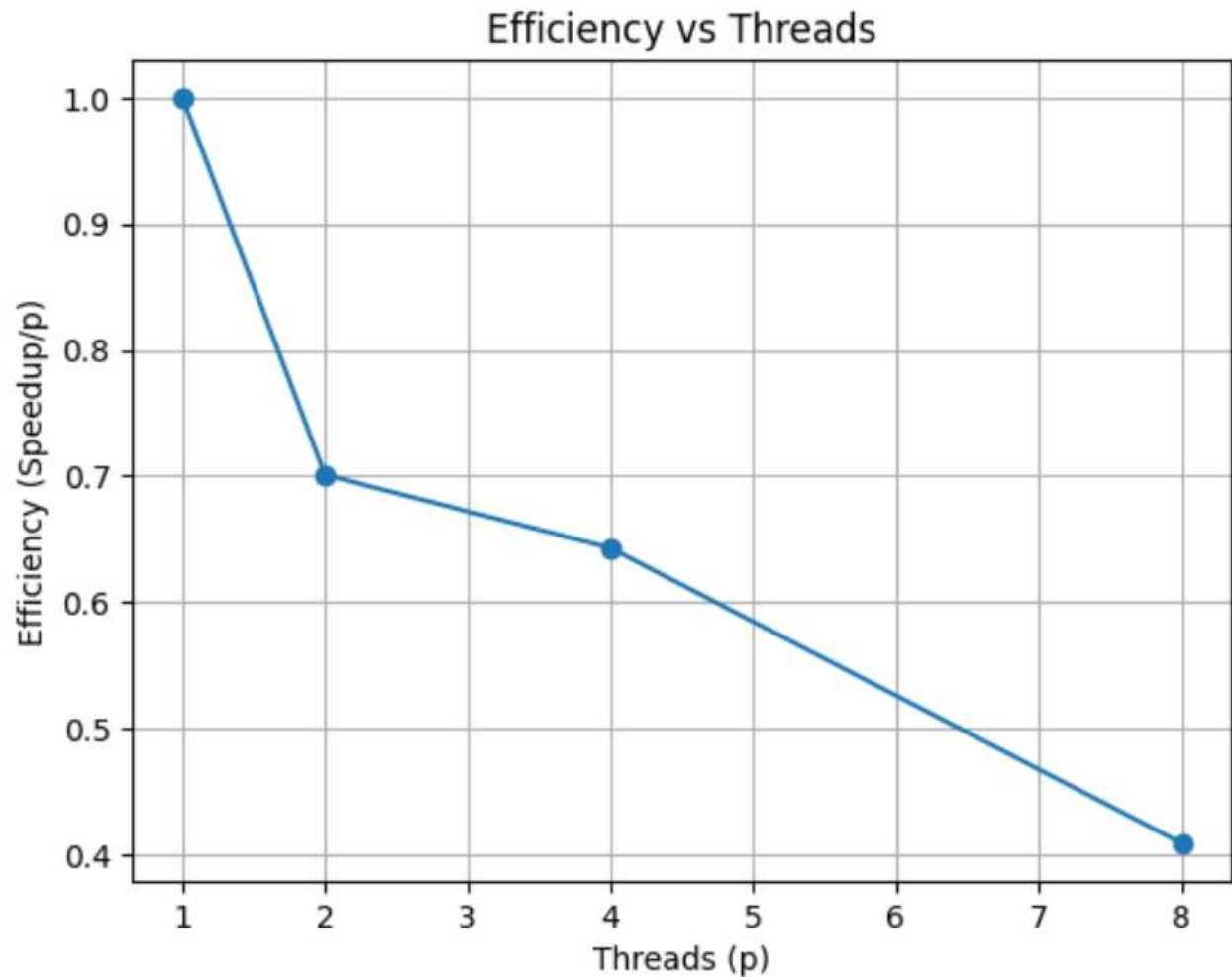
Metrics:

$$S = T_1 / T_p$$

$$S/P = E$$

Expected Behavior

- Speedup increases with more processes
- Efficiency decreases due to communication overhead
- Communication cost dominates when p is large



4.2 Weak Scaling

Problem size increases *proportionally* with number of processes:

- Rank 1 → 1 MB
- Rank 2 → 2 MB
- Rank 4 → 4 MB

Expected Behavior

- Execution time ideally remains constant
- In reality, time increases slightly due to:
 - MPI overhead
 - bandwidth limitations
 - synchronization

4.3 Interpretation

- Strong scaling suffers from Amdahl's Law
 - Weak scaling suffers from communication overhead
 - Performance decreases as message size increases
-

5. Latency & Bandwidth Benchmark (Member 4)

5.1 Overview

A Ping-Pong benchmark was implemented to measure:

- Latency (small messages)
- Bandwidth (large messages)

5.2 Method

For each message size:

1. Rank 0 sends message to Rank 1
2. Rank 1 sends same message back
3. Time measured with `MPI_Wtime`
4. Repeated **1000 times** for accuracy
5. CSV written in form:

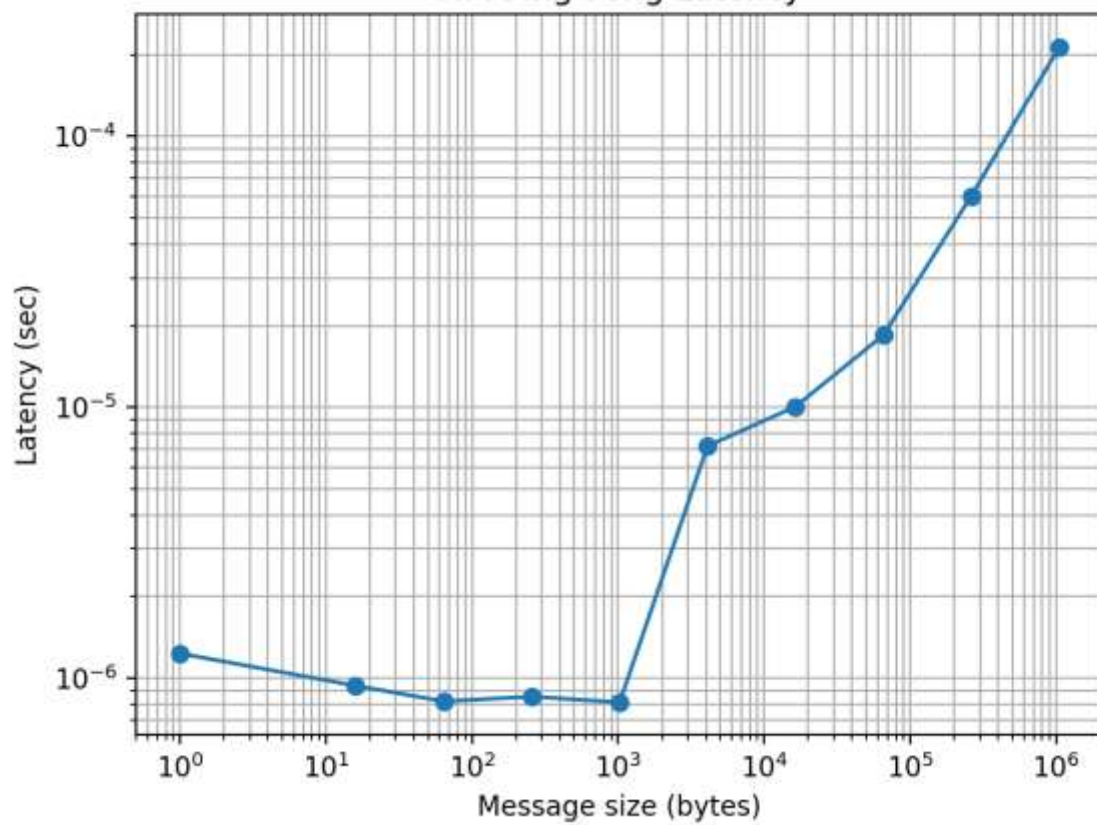
```
size, latency, bandwidth
```

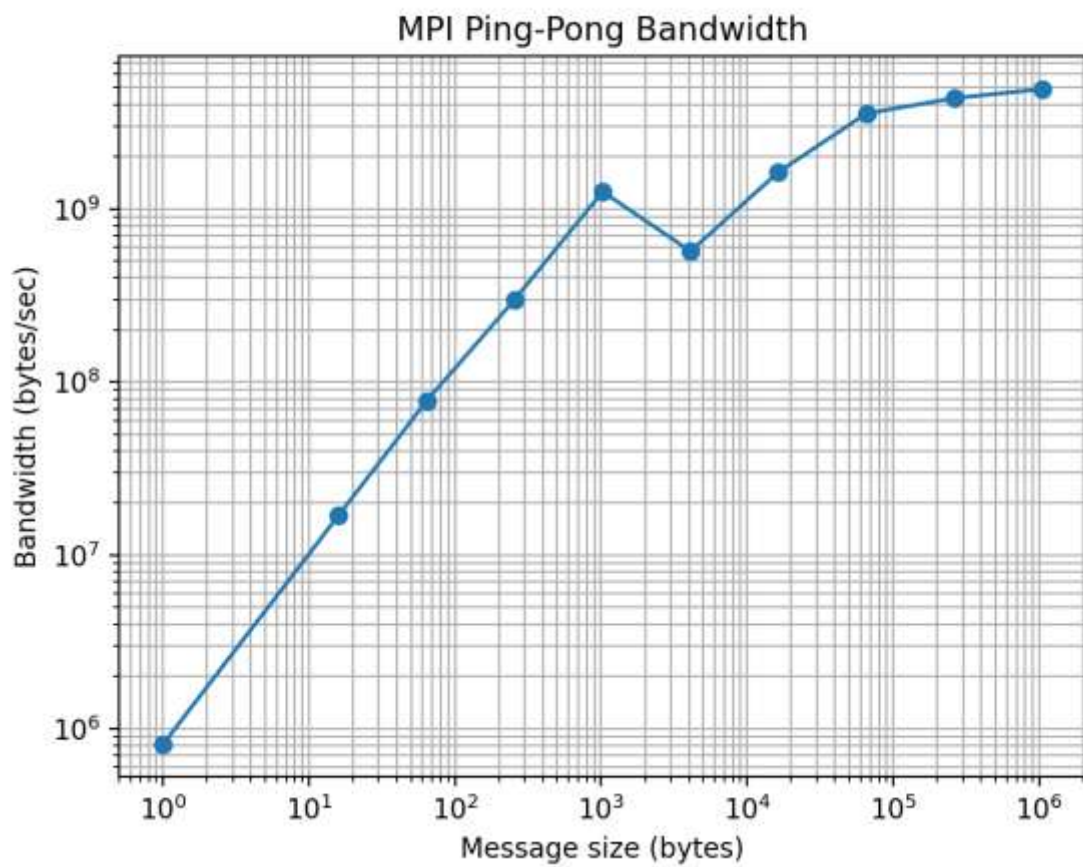
5.3 Results

Plots generated via `plot_pingpong.py`:

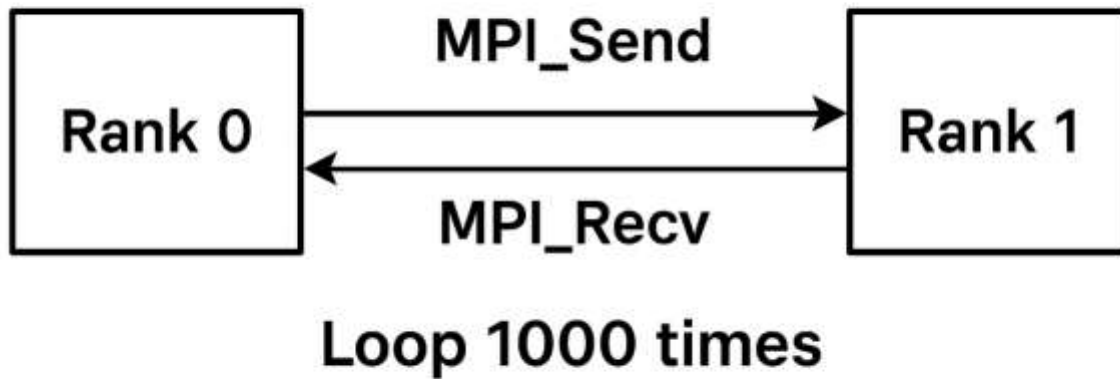
- **latency_plot.png**
- **bandwidth_plot.png**

MPI Ping-Pong Latency





5.4 Ping-Pong Communication Diagram



5.5 Interpretation

- Latency nearly constant for tiny messages
 - Bandwidth increases with size until reaching hardware saturation
 - These results explain why communication becomes the bottleneck as processes increase
-

6. Conclusion

- MPI provides efficient parallelism but communication costs become critical at scale
- Non-blocking communication improves performance significantly
- Strong scaling limited by Amdahl's law
- Weak scaling limited by communication and bandwidth
- Ping-Pong tests reveal the underlying cost of message passing
- Overall, combining all measurements gives a full picture of MPI performance